

HybridMPMCQueue: progettazione e valutazione sperimentale in MoStream

Documentazione tecnica per attività di tesi

Progetto MoStream - 22 luglio 2026

Questa relazione descrive HybridMPMCQueue, una coda bounded Multiple Producer, Multiple Consumer che combina un fast path basato su Compare-And-Swap con un fallback Fetch-And-Add attivato dopo un numero configurabile di collisioni.

L'obiettivo è conservare il comportamento CAS quando la contesa è bassa e ridurre i retry sul contatore condiviso quando la contesa cresce. Sono documentati algoritmo, memory ordering, API, correttezza osservata e confronto sperimentale delle soglie 1, 2, 4 e 8. I risultati mostrano che una soglia bassa è preferibile nei workload bilanciati e molto contesi, mentre alcuni workload sbilanciati favoriscono la soglia 2. La soluzione ibrida può superare CAS e, in alcuni casi asimmetrici, FAA pura; non domina però FAA nei casi bilanciati ad alta contesa e mantiene i rischi dei ticket irrevocabili quando usa il fallback.

Contesto e obiettivo

Le code esaminate nel progetto MoStream sono ring buffer bounded: la capacità è fissata alla costruzione e non cresce durante l'esecuzione. MPMC indica che più producer e più consumer possono operare contemporaneamente. CAS e FAA non definiscono la semantica della coda, ma il meccanismo usato per prenotare una posizione nel ring.

La coda CAS originale evita di avanzare il contatore finché la cella corrente non è disponibile, ma più thread possono contendersi lo stesso valore e fallire `compare_exchange`. La variante FAA assegna subito ticket distinti con `fetch_add`, eliminando i retry CAS, ma il ticket non può essere annullato.

HybridMPMCQueue cerca un punto intermedio:

- CAS come percorso iniziale di push e pop bloccanti;
- conteggio locale delle sole collisioni CAS effettive;
- fallback FAA dopo una soglia `compile-time`;
- CAS sempre usato da `try_push` e `try_pop`;
- nessun cambio globale di modalità e nessun contatore adattivo condiviso nel percorso prestazionale.

Struttura della coda

L'implementazione si trova in `MoStream/Hybrid_queue.mojo`. Il tipo principale è:

```
HybridMPMCQueue[T: Copyable, cas_failures_before_faa: Int = 4]
```

I campi fondamentali sono:

<code>buffer</code>	array bounded di <code>Cell[T]</code>
<code>size</code>	capacità del ring
<code>mask</code>	<code>size - 1</code>
<code>enqueue_pos</code>	prossimo ticket producer
<code>dequeue_pos</code>	prossimo ticket consumer
<code>enqueue_faa_fallbacks</code>	contatore diagnostico opzionale
<code>dequeue_faa_fallbacks</code>	contatore diagnostico opzionale

La capacità predefinita è 1024. Ogni valore ammesso deve essere una potenza di due e almeno pari a due. L'indice fisico è `ticket & mask`; una capacità non

valida produce un messaggio di errore e termina il processo.

enqueue_pos, dequeue_pos e i due contatori diagnostici sono racchiusi in PaddedAtomicU64, la cui dimensione è portata a 64 byte. Lo spacing è pensato per ridurre il false sharing fra contatori adiacenti. Il sorgente non impone però esplicitamente un allineamento di 64 byte all'oggetto contenitore: la sola dimensione non prova che ogni contatore inizi su una cache line. Layout e allineamento effettivi devono quindi essere verificati sul compilatore usato. Le celle del buffer sono compatte e non isolate singolarmente: HybridMPMCQueue non deve essere confusa con PaddedFAAQueue.

Protocollo dei sequence number

Ogni Cell contiene un sequence number atomico e un Optional[T]. Il sequence number rappresenta contemporaneamente stato e generazione della cella. Per un producer con ticket p:

- sequence uguale a p indica che la cella è libera per quella generazione;
- dopo la scrittura del payload, sequence viene impostato a p+1.

Per un consumer con ticket c:

- sequence uguale a c+1 indica che il payload è pubblicato;
- dopo l'estrazione, sequence viene impostato a c+size, rendendo la cella libera nel giro successivo del ring.

Questo protocollo distingue le generazioni che riusano lo stesso indice e permette di riconoscere una cella piena, vuota o non ancora pubblicata senza un flag globale aggiuntivo.

Push bloccante: CAS e fallback FAA

Ogni chiamata push mantiene un contatore locale cas_failures, inizializzato a zero. Il percorso può essere riassunto così:

1. carica enqueue_pos con ordering RELAXED;
2. individua la cella con ticket & mask;
3. attende sequence == ticket;
4. prova CAS(ticket, ticket + 1);
5. se il CAS riesce, pubblica il payload;
6. se il CAS perde una collisione, incrementa cas_failures;
7. raggiunta la soglia, usa fetch_add e attende la cella del nuovo ticket;
8. pubblica con store RELEASE sul sequence number.

Una cella non disponibile non incrementa cas_failures. Una queue piena, o una posizione già prenotata ma non ancora pubblicata, è un problema di disponibilità e non viene interpretata come contesa CAS. Il fallback si attiva soltanto quando la cella osservata era pronta ma un altro producer ha modificato enqueue_pos prima del compare_exchange.

CAS e FAA operano sullo stesso oggetto atomico. La loro serializzazione assegna posizioni distinte: un CAS con valore expected ormai superato fallisce, mentre fetch_add restituisce il valore che precedeva il proprio incremento. La correttezza complessiva dipende anche dal protocollo per-cell sequence.

Pop bloccante

Il lato consumer è simmetrico. pop attende sequence == ticket+1, prova a incrementare dequeue_pos mediante CAS e, dopo il numero configurato di collisioni, prenota una posizione con fetch_add. Il payload viene estratto da Optional e la cella viene liberata con una store RELEASE di ticket+size.

La soglia è unica per producer e consumer. L'implementazione corrente non consente quindi di scegliere, per esempio, soglia enqueue 1 e soglia dequeue 2. Questa estensione è interessante per topologie asimmetriche, ma richiede una variante dell'interfaccia compile-time.

Significato delle soglie 1, 2, 4 e 8

Il parametro `cas_failures_before_faa` è almeno 1. Il fallback avviene dopo quel numero di CAS falliti nella singola operazione bloccante:

`tab( ), box;`

Soglia Comportamento Compromesso atteso

1 FAA dopo la prima collisione reazione rapida alla contesa

2 FAA dopo due collisioni fast path CAS leggermente più persistente

4 valore predefinito corrente politica conservativa

8 FAA dopo otto collisioni costo elevato se la contesa persiste

Il conteggio è locale alla chiamata: non aggiunge RMW condivisi e non influenza la chiamata successiva. La coda non apprende una modalità globale e non cambia algoritmo mentre esistono ticket pendenti. Di conseguenza, il termine ibrida descrive il percorso della singola operazione, non un selettore runtime per topologia.

Operazioni non bloccanti

`try_push` e `try_pop` usano sempre load, verifica del sequence number e un solo compare_exchange. Se la cella non è pronta o il CAS viene perso, ritornano immediatamente. Un `fetch_add` non sarebbe adatto: il fallimento immediato lascerebbe un ticket non completato e quindi un buco nel flusso delle posizioni.

In questa API `try_push` restituisce `Optional[T]`: `None` indica inserimento riuscito, mentre il payload restituito indica che l'elemento non è stato inserito.

`try_pop` restituisce `None` se non completa l'estrazione. Sotto contesa il ritorno negativo può derivare anche dalla perdita della gara CAS, non soltanto da uno stato pieno o vuoto rimasto stabile.

Memory ordering

Le operazioni sui contatori `enqueue_pos` e `dequeue_pos` usano `Ordering.RELAXED`. La sincronizzazione del payload è demandata ai sequence number:

- il producer scrive data e pubblica sequence con `RELEASE`;
- il consumer osserva sequence con `ACQUIRE` prima di leggere data;
- il consumer libera la cella con `RELEASE`;
- il producer della generazione seguente osserva la liberazione con `ACQUIRE`.

Separare assegnazione del ticket e pubblicazione del dato consente ai contatori di rimanere relaxed. Ridurre anche l'ordinamento dei sequence number romperebbe invece la relazione happens-before necessaria alla visibilità del payload.

Politica di attesa

`push` e `pop` sono operazioni bloccanti a spin: se la cella attesa non ha il sequence previsto, il thread continua a controllarla. Ogni 1024 osservazioni senza successo viene chiamato `sleep(0.0)`, che offre allo scheduler un'occasione di eseguire altri thread. Lo stesso meccanismo viene usato prima e dopo il fallback FAA.

Questa politica cerca un compromesso tra latenza e consumo CPU, ma 1024 non è una costante universalmente ottimale. Core dedicati, oversubscription, durata

degli stage e politica dello scheduler possono cambiare il risultato.

API Mojo

Esempi con soglia predefinita e soglia esplicita:

```
from MoStream.Hybrid_queue import HybridMPMCQueue

var default_queue = HybridMPMCQueue[Int](1024)          # soglia 4
var quick_fallback = HybridMPMCQueue[Int, 1](1024)      # soglia 1

quick_fallback.push(42)
var value = quick_fallback.pop()
```

Metodi pubblici:

- push(item): inserimento bloccante CAS con possibile fallback FAA;
- pop(): estrazione bloccante CAS con possibile fallback FAA;
- try_push(item): tentativo CAS, con restituzione del payload non inserito;
- try_pop(): tentativo CAS, con risultato Optional[T];
- estimated_len(): stima relaxed, limitata all'intervallo 0..capacity;
- enqueue_fallback_count e dequeue_fallback_count: diagnostica opzionale.

estimated_len non è una misura linearizzabile. Letture concorrenti e ticket prenotati ma non ancora pubblicati possono far divergere la stima dal numero di payload immediatamente consumabili.

Diagnostica dei fallback

Il sorgente contiene ENABLE_FALLBACK_DIAGNOSTICS=False. Quando l'opzione è disattivata, i due contatori restano a zero anche se il percorso FAA viene eseguito. Ciò evita una fetch_add condivisa aggiuntiva proprio nel ramo che si vuole misurare. Pertanto un valore diagnostico zero nella build prestazionale non dimostra l'assenza di fallback.

Per studiare la frequenza del ramo occorre una build diagnostica separata e confrontabile, accettando che il conteggio perturbi il throughput. I risultati prestazionali e quelli diagnostici non dovrebbero essere raccolti nella stessa esecuzione.

Test di correttezza presenti

Test sequenziale

Tests/test_hybrid_queue.mojo crea una queue di capacità 8, la riempie con try_push, verifica il rifiuto del nono elemento, controlla l'ordine FIFO, verifica try_pop su queue vuota ed esegue 32 cicli push/pop per attraversare più volte il wrap-around del ring.

Test concorrente

Tests/test_hybrid_queue_concurrent.mojo usa 4 producer, 4 consumer e 250000 messaggi per producer, per un totale di un milione di valori. I producer pubblicano intervalli disgiunti; l'ultimo inserisce un sentinel per consumer. Ogni consumer mantiene conteggio e checksum locali. Dopo il join vengono confrontati:

```
conteggio atteso = 1000000
checksum atteso  = N x (N - 1) / 2
```

Entrambi i test sono stati ricompilati ed eseguiti il 22 luglio 2026 e hanno restituito PASS. Il test concorrente ha stampato contatori fallback pari a zero perché la diagnostica compile-time è disattivata. I test osservano perdita,

duplicazione, FIFO sequenziale e wrap-around, ma non sostituiscono una prova formale né esplorano esaustivamente tutti gli interleaving.

Metodologia del benchmark delle soglie

La matrice in results_hybrid_thresholds.csv comprende le topologie 1P-1C, 2P-2C, 4P-4C, 8P-8C, 1P-8C e 8P-1C, ciascuna con capacità 16, 1024 e 65536. Ogni producer invia 250000 interi e ogni configurazione viene ripetuta 10 volte. Sono confrontate CAS, FAA e Hybrid con soglie 1, 2, 4 e 8.

Conteggio e checksum sono locali ai consumer durante la regione temporizzata. Il runner cambia l'ordine delle implementazioni tra le ripetizioni per ridurre il bias d'ordine. Ogni campione è accettato soltanto con valid=True. Per ogni serie sono calcolati throughput medio, deviazione standard campionaria e intervallo di confidenza bilaterale al 95% con Student-t.

La campagna comprende 18 configurazioni per 10 ripetizioni: 720 esecuzioni Hybrid e 360 esecuzioni di riferimento CAS/FAA. Tutte hanno superato conteggio e checksum. Il throughput è espresso in milioni di messaggi al secondo.

Ambiente sperimentale

```
tab( ), box;
Componente Valore
CPU Intel Xeon Gold 5512U
Topologia 1 socket, 28 core, 2 thread/core, 56 CPU logiche
Architettura x86_64, 1 nodo NUMA
Sistema Linux 5.15.0-168-generic
Mojo 1.0.0b1, revisione a9591de6
Payload Int
```

I risultati appartengono a questa macchina e a questa versione del software. Non sono automaticamente trasferibili ad altre microarchitetture, compilatori o politiche di scheduling.

Risultati: topologie bilanciate

La tabella riporta il throughput medio. H1, H2, H4 e H8 indicano la soglia di collisioni prima del fallback FAA.

```
tab( ), box;
Topologia Capacità CAS FAA H1 H2 H4 H8
1P-1C 16 4.039 5.080 4.448 4.767 4.783 4.761
1P-1C 1024 4.667 5.887 5.622 6.431 6.400 6.353
1P-1C 65536 3.931 6.044 4.065 4.550 4.488 4.561
2P-2C 16 3.949 5.939 5.083 4.171 3.818 3.611
2P-2C 1024 4.664 7.272 5.564 4.824 4.465 4.358
2P-2C 65536 5.433 9.433 7.480 6.239 5.665 5.458
4P-4C 16 3.510 8.336 6.821 4.873 3.958 3.631
4P-4C 1024 3.875 8.597 7.301 5.057 4.183 3.900
4P-4C 65536 3.556 8.998 6.780 4.843 3.944 3.619
8P-8C 16 2.645 9.520 7.287 4.507 3.262 2.753
8P-8C 1024 2.866 10.008 7.616 4.758 3.502 3.009
8P-8C 65536 2.715 8.526 7.140 4.411 3.176 2.784
```

Con almeno 2 producer e 2 consumer H1 è sempre la migliore soglia Hybrid. Nelle configurazioni 4P-4C raggiunge circa 1.89-1.96x CAS e il 75-85% del throughput FAA. In 8P-8C raggiunge circa 2.63-2.76x CAS e il 76-84% di FAA. FAA pura rimane più veloce in tutti questi casi bilanciati.

Aumentare la soglia degrada quasi monotonicamente i casi contesi: il thread paga più CAS falliti prima di eseguire comunque fetch_add. Con 8P-8C, H8 resta solo circa 1.03-1.05x CAS e raggiunge circa il 29-33% di FAA.

Nel caso 1P-1C non esiste contesa fra producer sul contatore enqueue né fra consumer su dequeue. Le quattro soglie percorrono quindi normalmente lo stesso fast path; le differenze fra H1, H2, H4 e H8 non devono essere attribuite al fallback e possono riflettere rumore, ordine, scheduling o layout del codice.

Risultati: topologie sbilanciate

```
tab( ),box;
Topologia Capacità CAS FAA H1 H2 H4 H8
1P-8C 16 3.473 4.061 3.791 3.824 4.790 3.904
1P-8C 1024 3.695 4.370 4.678 6.778 4.848 4.067
1P-8C 65536 4.037 4.323 4.753 7.000 5.249 4.372
8P-1C 16 3.970 5.384 6.034 4.900 4.005 3.691
8P-1C 1024 3.952 5.156 5.323 5.746 4.442 4.099
8P-1C 65536 4.350 6.000 9.941 6.378 4.925 4.419
```

Nei casi 1P-8C con capacità 1024 e 65536, H2 ottiene rispettivamente 6.778 e 7.000 Mmsg/s, circa 1.55x e 1.62x FAA nei rapporti medi appaiati. Con capacità 16 il valore migliore è invece H4; questo singolo risultato non giustifica una regola generale senza ulteriori repliche e analisi della latenza.

Con 8P-1C H1 è migliore per capacità 16 e 65536, mentre H2 prevale a 1024. H1 raggiunge 9.941 Mmsg/s a capacità 65536, circa 1.64x FAA e 2.26x CAS nei rapporti medi appaiati. Questi risultati mostrano il potenziale del fast path CAS sul lato non conteso e del fallback rapido sul lato con molti partecipanti.

Interpretazione delle soglie

La soglia 4 predefinita è conservativa ma non è quella con il miglior throughput generale nei dati raccolti. Una politica iniziale basata sui risultati sarebbe:

```
2P-2C, 4P-4C, 8P-8C    soglia 1
molti producer, 1 C     soglia 1, da validare per capacità
1 producer, molti C     soglia 2, con eccezione osservata a capacità 16
1P-1C                   soglia non causale in assenza di collisioni
```

Questa regola non deve essere codificata come verità universale: è derivata dalla stessa matrice usata per valutarla e rischia overfitting. Per una tesi è opportuno definire la politica su una parte dei workload e validarla su nuove topologie, payload, capacità e macchine.

Garanzie di progresso e head-of-line blocking

push e pop possono attendere quando la queue è piena o vuota; la coda non è wait-free. Dopo il fallback, fetch_add assegna un ticket irrevocabile. Se un thread viene sospeso dopo la prenotazione e prima della pubblicazione o del rilascio della cella, altri partecipanti possono raggiungere una posizione che dipende da quel ticket e restare in attesa: è head-of-line blocking.

Anche un percorso CAS che ha già riservato con successo una posizione dipende dal completamento del thread proprietario. La differenza importante è che il fallback FAA può prenotare la posizione corrente senza verificarne nuovamente la disponibilità e può quindi aspettare uno slot futuro. La coda ibrida limita l'esposizione al ramo FAA alle operazioni che hanno osservato collisioni, ma non elimina il problema.

Non sono stati ancora misurati thread deliberatamente sospesi, oversubscription, fairness o starvation. Il throughput osservato quando tutti i worker progrediscono non costituisce una garanzia di progresso in condizioni avverse.

Limiti sperimentali

- Una sola macchina, un solo sistema operativo e una sola versione Mojo.
- Dieci ripetizioni non eliminano rumore sistematico, autocorrelazione o effetti di frequenza e temperatura.
- È stato misurato throughput con payload Int, non latenza p50/p95/p99.
- Non sono inclusi consumo CPU, energia, traffico cache, miss TLB o CAS falliti misurati da contatori hardware.
- Non sono stati provati payload grandi, capacità non power-of-two o overflow dei contatori UInt64.
- I test count/checksum rilevano perdita e duplicazione, ma non provano la linearizzabilità per ogni interleaving.
- I risultati sono microbenchmark e non misurano direttamente l'effetto end-to-end sulle pipeline MoStream.

Integrazione in MoStream

HybridMPMCQueue è attualmente usata dai test e dal benchmark, ma non è importata dal runtime MoStream. Per integrarla senza compromettere il confronto occorre:

1. rendere l'interfaccia compatibile con il punto di costruzione del Communicator;
2. selezionare la soglia come specializzazione compile-time, evitando branch per ogni messaggio;
3. mantenere try_push e try_pop sul percorso CAS;
4. eseguire test end-to-end, shutdown, backpressure e sospensione dei worker;
5. misurare throughput e latenza su pipeline reali oltre al microbenchmark.

Se producer e consumer sono noti alla costruzione, una possibile evoluzione è usare soglie separate enqueue/dequeue. Una seconda evoluzione è selezionare una queue specializzata per topologia fuori dal percorso caldo, invece di fare dispatch runtime dentro push e pop.

Conclusioni

HybridMPMCQueue implementa correttamente, nei test osservati, un fast path CAS con fallback FAA basato su collisioni reali. Il progetto evita di confondere queue piena o vuota con contesa e conserva CAS per le operazioni try, nelle quali un ticket deve poter essere abbandonato.

La soglia 1 è la scelta Hybrid più efficace nelle topologie bilanciate con almeno 2 producer e 2 consumer: si avvicina a FAA e supera nettamente CAS quando la contesa cresce. Nei workload sbilanciati la soluzione ibrida può superare anche FAA, con H2 particolarmente efficace in 1P-8C a capacità medio-grande. La soglia 4 predefinita non è sostenuta come valore prestazionale generale dalla matrice attuale.

La conclusione prudente per la tesi è che l'ibridazione è utile quando il carico è variabile o asimmetrico, ma non produce un algoritmo universalmente dominante. FAA pura resta superiore nei casi bilanciati più contesi; inoltre il fallback mantiene ticket irrevocabili e head-of-line blocking. Prima dell'adozione nel runtime sono necessari test di latenza, progresso avverso e integrazione reale.

File reproducibili

```
MoStream/Hybrid_queue.mojo
Tests/test_hybrid_queue.mojo
Tests/test_hybrid_queue_concurrent.mojo
Benchmarks/QueueBenchmark/queue_benchmark.mojo
```

Benchmarks/QueueBenchmark/run_suite.py
Benchmarks/QueueBenchmark/results_hybrid_thresholds.csv